

PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA



ASIGNATURA:

ASEGURAMIENTO DE CALIDAD DE SOFTWARE

GRUPO:

1890-4346

PRESENTADO POR:

BRIGIBEL YSAURA MARTÍNEZ MARTÍNEZ, 1014-4327

STARLIN JOSÉ FRIAS LUNA, 1014-3611

TEMA:

PLAN DE PROYECTO Y REQUISITOS

PRESENTADO A:

ING. FREDDY A. PEÑA

FECHA DE ENTREGA:

JUEVES 3 DE JULIO DEL 2025

SANTIAGO, REPÚBLICA DOMINICANA

Plan de Proyecto – Sistema de Gestión de Inventarios con QAS

Alcance:

Desarrollar un sistema web que permita gestionar productos en el inventario de una pequeña empresa, con operaciones de alta calidad: agregar, editar, eliminar y visualizar productos. Incluir autenticación basada en JWT, control de acceso para administrador, y compatibilidad con distintos navegadores mediante pruebas automatizadas.

Objetivos:

- Crear un sistema que permita CRUD completo de productos.
- Asegurar la calidad con pruebas de aceptación y compatibilidad.
- Proveer una API REST segura con Spring Boot.
- Desarrollar una interfaz moderna y responsiva con React/Next.js.
- Contenerizar el sistema para facilitar su despliegue en distintos entornos.

Entregables:

- Código fuente del backend (API REST) y frontend.
- Base de datos con migraciones gestionadas por Flyway.
- Contenedor Docker funcional.
- Documentación de requisitos y pruebas.
- Informe de gestión de riesgos.

Cronograma detallado

- **Semana 1 (29 mayo – 4 junio)**
 - Configurar el repositorio Git y el entorno de desarrollo.
 - Implementar autenticación JWT en el backend.
 - Crear base de datos con tablas iniciales y configuración de migraciones con Flyway.
 - Configuración inicial de Prometheus y Grafana.

- **Semana 2 (5 – 11 junio)**
 - Desarrollar endpoints CRUD en el backend para la gestión de productos.
 - Probar funcionalidad básica de la API con herramientas como Postman.
 - Preparar script para contenedorización con Docker.
- **Semana 3 (12 – 18 junio)**
 - Crear el frontend: diseño de la pantalla de login y flujo de autenticación.
 - Implementar la vista principal con tabla de productos, búsqueda y filtros.
 - Configurar rutas protegidas en el frontend.
- **Semana 4 (19 – 25 junio)**
 - Completar las vistas de agregar y editar productos en el frontend.
 - Probar integración completa frontend-backend.
 - Iniciar redacción de la documentación de requisitos.
 - Agregar dashboards para métricas a Grafana.
 - Creación de pruebas unitarias y de coberturas con Jacoco Report.
- **Semana 5 (26 junio – 3 julio)**
 - Realizar pruebas E2E con Playwright en diferentes navegadores.
 - Configuración y realización de pruebas con Cucumber para funcionalidades automatizadas.
 - Terminar documentación (plan de proyecto, gestión de riesgos, requisitos).
 - Revisar, corregir errores y preparar la entrega final.

Requisitos funcionales

- RF1: El sistema debe permitir al administrador iniciar sesión mediante un formulario de login.
- RF2: El sistema debe permitir agregar productos con nombre, descripción, categoría, precio y cantidad.
- RF3: El sistema debe permitir editar información de productos existentes.
- RF4: El sistema debe permitir eliminar productos del inventario.
- RF5: El sistema debe mostrar un listado de productos con búsqueda y filtros por categoría.

- RF6: Solo usuarios autenticados podrán acceder a las funcionalidades anteriores.
- RF7: La API REST debe permitir integrar el sistema con otras aplicaciones vía endpoints seguros.
- RF8: El sistema debe permitir los monitoreos de las llamadas HTTP mediante Grafana.

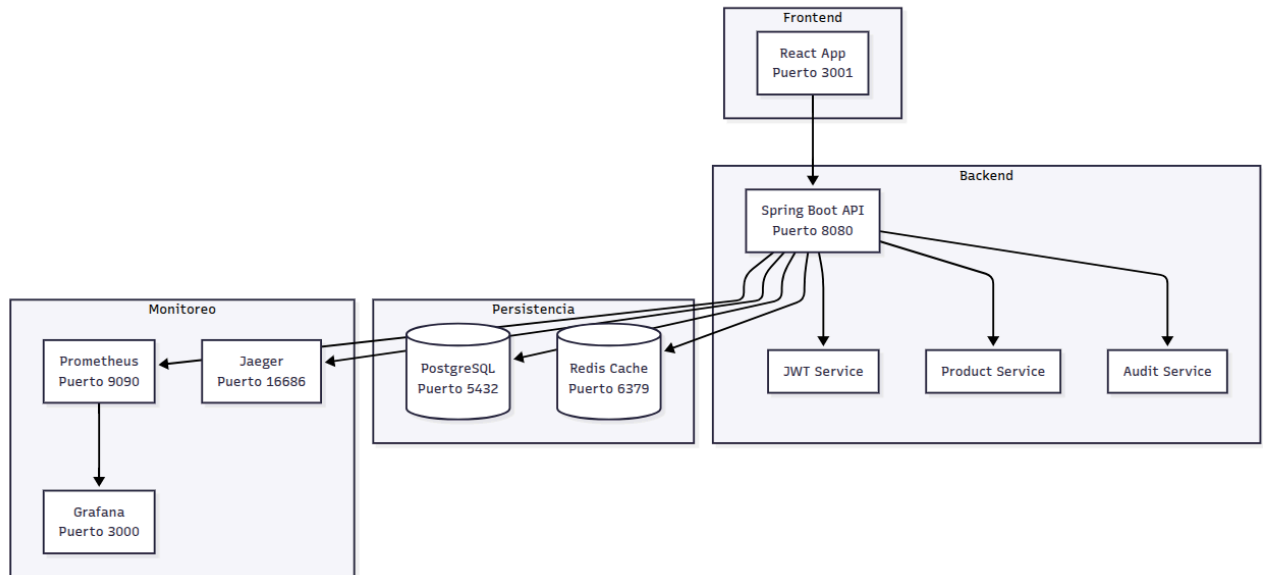
Requisitos no funcionales

- RNF1: El sistema debe ser compatible con los navegadores modernos (Chrome, Firefox, Edge, Safari).
- RNF2: El tiempo de respuesta del servidor no debe exceder 2 segundos en operaciones CRUD.
- RNF3: La interfaz debe adaptarse correctamente a dispositivos móviles y de escritorio (diseño responsivo).
- RNF4: Debe desarrollarse siguiendo prácticas de aseguramiento de calidad: pruebas automatizadas con Playwright y revisiones de código.
- RNF5: El sistema debe soportar autenticación JWT con almacenamiento seguro del token en cookies o almacenamiento local.

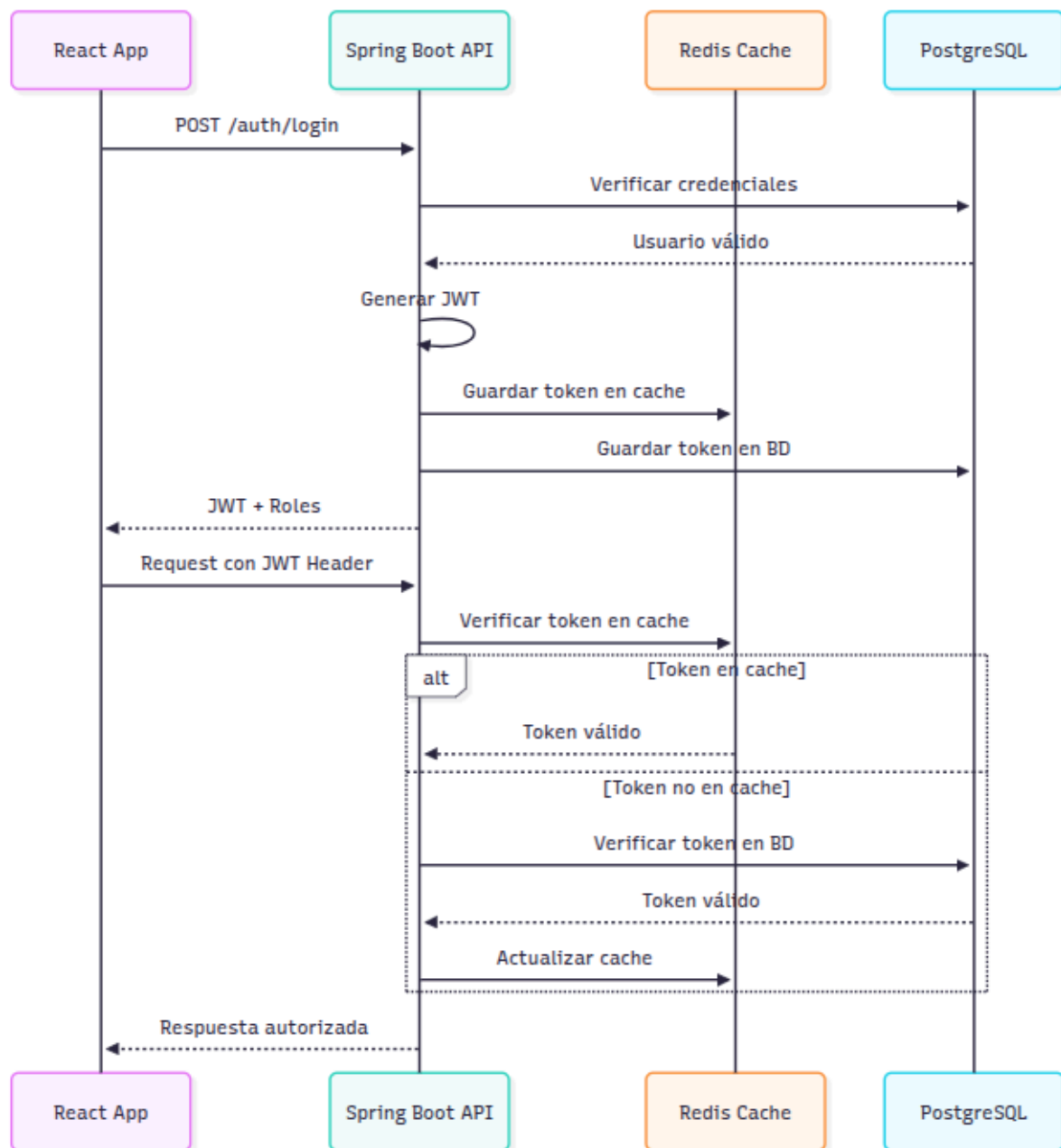
Documentación Técnica - Sistema de Gestión de Inventarios

1. Arquitectura del Sistema

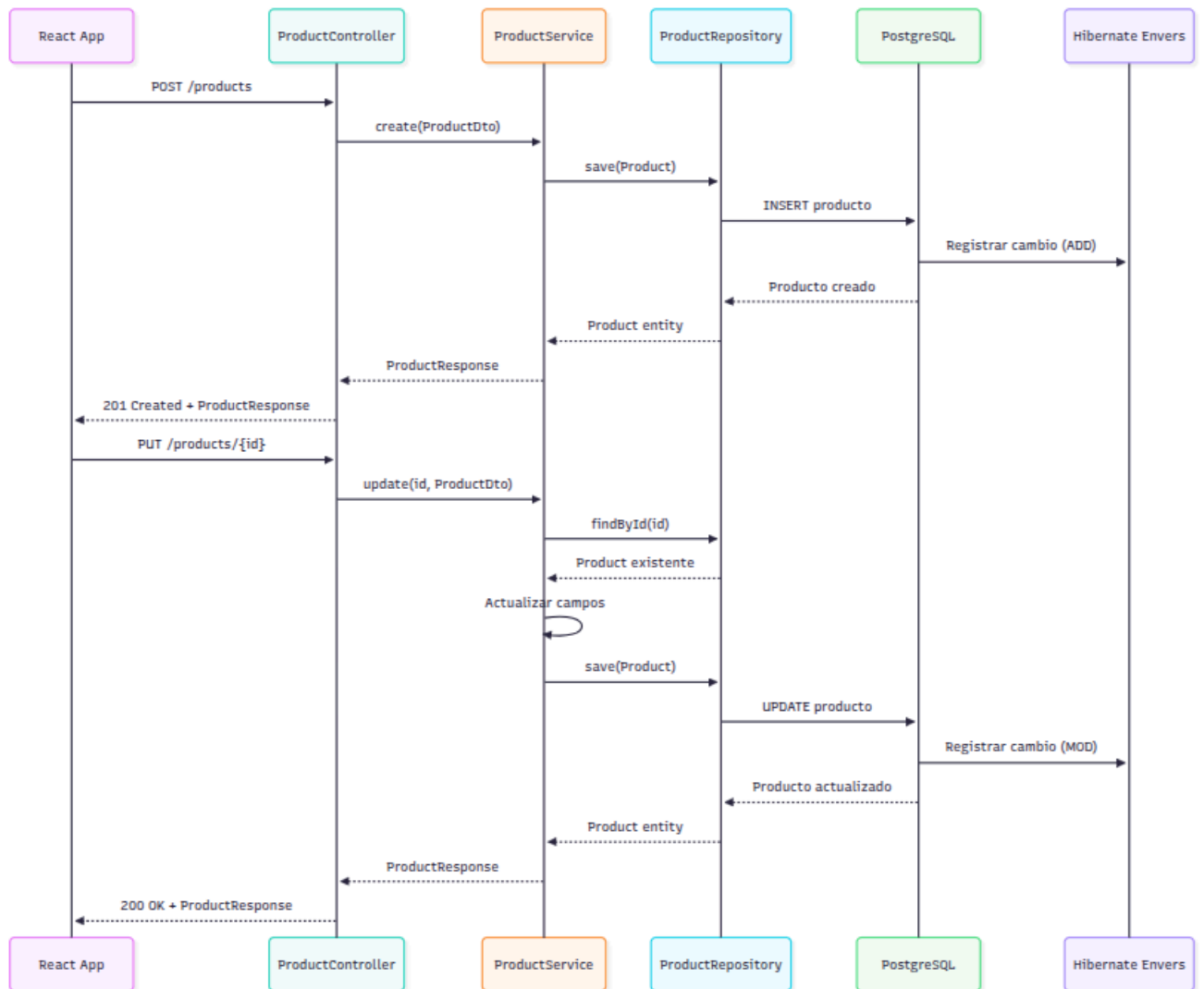
1.1 Diagrama de Arquitectura General



1.2 Flujo de Autenticación



1.3 Flujo CRUD de Productos



2. Guía de Instalación

2.1 Instalación con Docker (Recomendado)

Prerrequisitos:

- Docker & Docker Compose
- Git

Pasos:

1. Clonar repositorio:

- `git clone [repo-url]`
- `cd gestion-inventario`

2. Configurar variables de entorno:

- `cat > .env << EOF`
- `SPRING_DATASOURCE_USERNAME=postgres`
- `SPRING_DATASOURCE_PASSWORD=postgres`
- `JWT_SECRET=mySecretKey123456789012345678901234567890`
- `JWT_EXPIRATION=86400000`
- `EOF`

3. Descargar OpenTelemetry:

- `mkdir -p otel`
- `wget https://github.com/open-telemetry/opentelemetry-java-instrumentation/releases/latest/download/opentelemetry-javaagent.jar -O otel/opentelemetry-javaagent.jar`

4. Ejecutar sistema:

- `docker compose up --build -d`

5. Verificar servicios:

- Backend: <http://localhost:8080/actuator/health>
- Frontend: <http://localhost:3001>
- Grafana: <http://localhost:3000> (admin/admin)

3. APIs Principales

- POST /auth/login - Iniciar sesión
- POST /auth/validate - Validar token
- POST /auth/logout - Cerrar sesión

3.2 Productos

- GET /products - Listar con filtros
- POST /products - Crear producto
- GET /products/{id} - Obtener por ID
- PUT /products/{id} - Actualizar
- DELETE /products/{id} - Eliminar

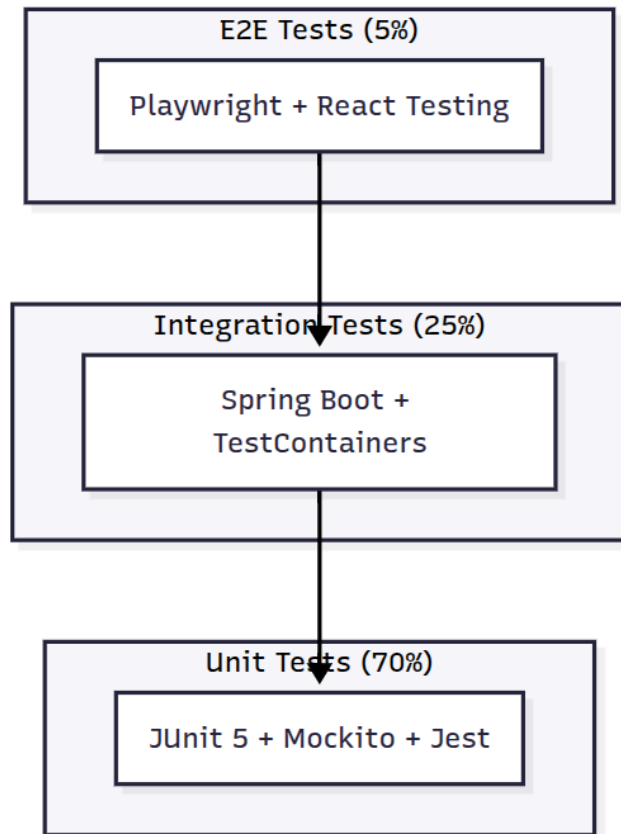
3.3 Auditoría

- GET /product-audit/product/{id}/history - Historial producto
- GET /product-audit/all/history - Historial completo

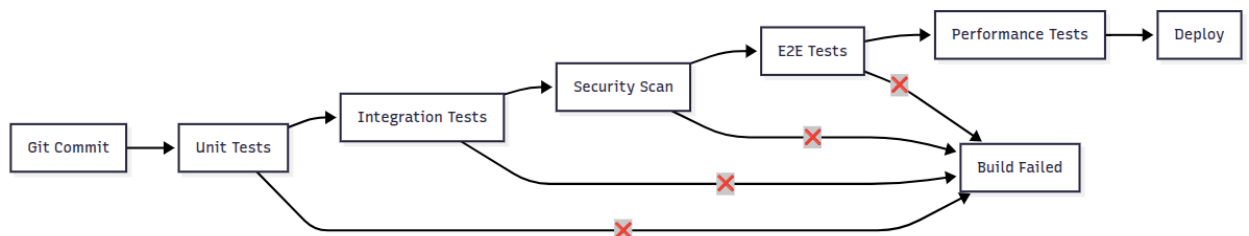
Guía de Pruebas - Sistema de Gestión de Inventarios

1. Estrategia de Pruebas

1.1 Pirámide de Pruebas



1.2 Flujo de Pruebas CI/CD



2. Casos de Prueba Principales

2.1 Pruebas Unitarias - Backend

ID Componente	Descripción	Estado	Cobertura
TC001	ProductService.create(): Creación de producto válido	✓ APROBADO ▾	100% ▾
TC002	ProductService.create(): Validación de campos obligatorios	✓ APROBADO ▾	100% ▾
TC003	ProductService.findById(): Búsqueda de producto existente	✓ APROBADO ▾	100% ▾
TC004	ProductService.findById(): Producto inexistente	✓ APROBADO ▾	100% ▾
TC005	JwtServices.validateToken(): Token válido en Redis	✓ APROBADO ▾	100% ▾
TC006	JwtServices.validateToken(): Token inválido/expirado	✓ APROBADO ▾	100% ▾
TC007	ProductSpecification: Filtros de búsqueda	✓ APROBADO ▾	95% ▾

3. Defectos Encontrados

3.1 Defectos Críticos

DEF-001: Rate Limiting No Implementado

- **Severidad:** ● Alta
- **Prueba:** TC034
- **Estado:** Abierto
- **Impacto:** Vulnerabilidad a ataques DDoS

Solución propuesta:

```
@Component
public class RateLimitingFilter {
    @Autowired
    private RedisTemplate<String, String> redisTemplate;

    public boolean isAllowed(String clientId) {
        // Implementar sliding window
    }
}
```